АВТОМАТИЗИРОВАННАЯ ИНФОРМАЦИОННАЯ СИСТЕМА ДЛЯ УПРАВЛЕНИЯ КЛИЕНТАМИ СПОРТИВНОГО КЛУБА

ПРИЛОЖЕНИЕ 3. МОДЕЛИ

Пермь 2026

Содержание

1. ER-диаграмма база данных 3
2. Диаграмма использования (Use Case Diagram) 6
3. Контекстная диаграмма системы (DFD Level 0) 8
4. Диаграмма декомпозиции процессов (DFD Level 1) 10
5. Диаграмма последовательностей 15
   5.1. Диаграмма последовательностей «Просмотр списка клиентов» 15
   5.2. Диаграмма последовательности «Авторизация пользователя» 17
   5.3. Диаграмма последовательности «Добавление клиента» 19
   5.4. Диаграмма последовательности «Добавление тренера» 21
   5.5. Диаграмма последовательности «Добавление абонемента» 23
   5.6. Диаграмма последовательности «Добавление тренировки» 25
   5.7. Диаграмма последовательности «Добавление посещения» 28

1. ER-диаграмма база данных

ER-диаграмма отражает структуру информационной модели системы и показывает основные сущности, их атрибуты и взаимосвязи (рис. 1).

В системе используются следующие ключевые таблицы:

- users — содержит данные учетных записей для входа в систему (логин, хэш пароля, роль, статус активности).
- reports — хранит сведения о сформированных отчетах (название отчета, дата формирования).
- clients — содержит информацию о клиентах спортклуба (ФИО, дата рождения, телефон, e-mail, дата регистрации, примечания).
- trainers — хранит данные о тренерах (ФИО, контакты, специализация, дата найма, активность).
- memberships — таблица абонементов клиентов (тип абонемента, даты начала/окончания, цена, общее число посещений, оставшиеся посещения, статус).
- trainer_clients — связывает тренеров и клиентов (кто кому назначен, дата назначения, признак «основной тренер»).
- trainingsessions — хранит расписание и историю тренировок (тип тренировки, дата/время, длительность, статус, комментарий, ссылки на тренера и клиента).
- visits — фиксирует посещения клуба (дата посещения, примечание, ссылка на клиента, при необходимости — ссылка на абонемент и тренера).

Основные связи:

- один User может сформировать несколько Reports;
- один Client может иметь несколько Memberships;
- один Client может иметь несколько записей Visits;
- один Membership может быть связан с несколькими Visits (посещение может быть как по абонементу, так и без него — при разовом визите);
- один Trainer может быть связан с несколькими TrainingSessions, и один Client также может иметь несколько TrainingSessions;
- один Trainer может быть назначен нескольким Clients, и один Client может быть назначен нескольким Trainers через таблицу trainer_clients (связь многие-ко-многим).

Диаграмма обеспечивает наглядное представление логики хранения данных и подтверждает целостность структуры, используемой системой учета клиентов, тренеров, абонементов, посещений и тренировок.

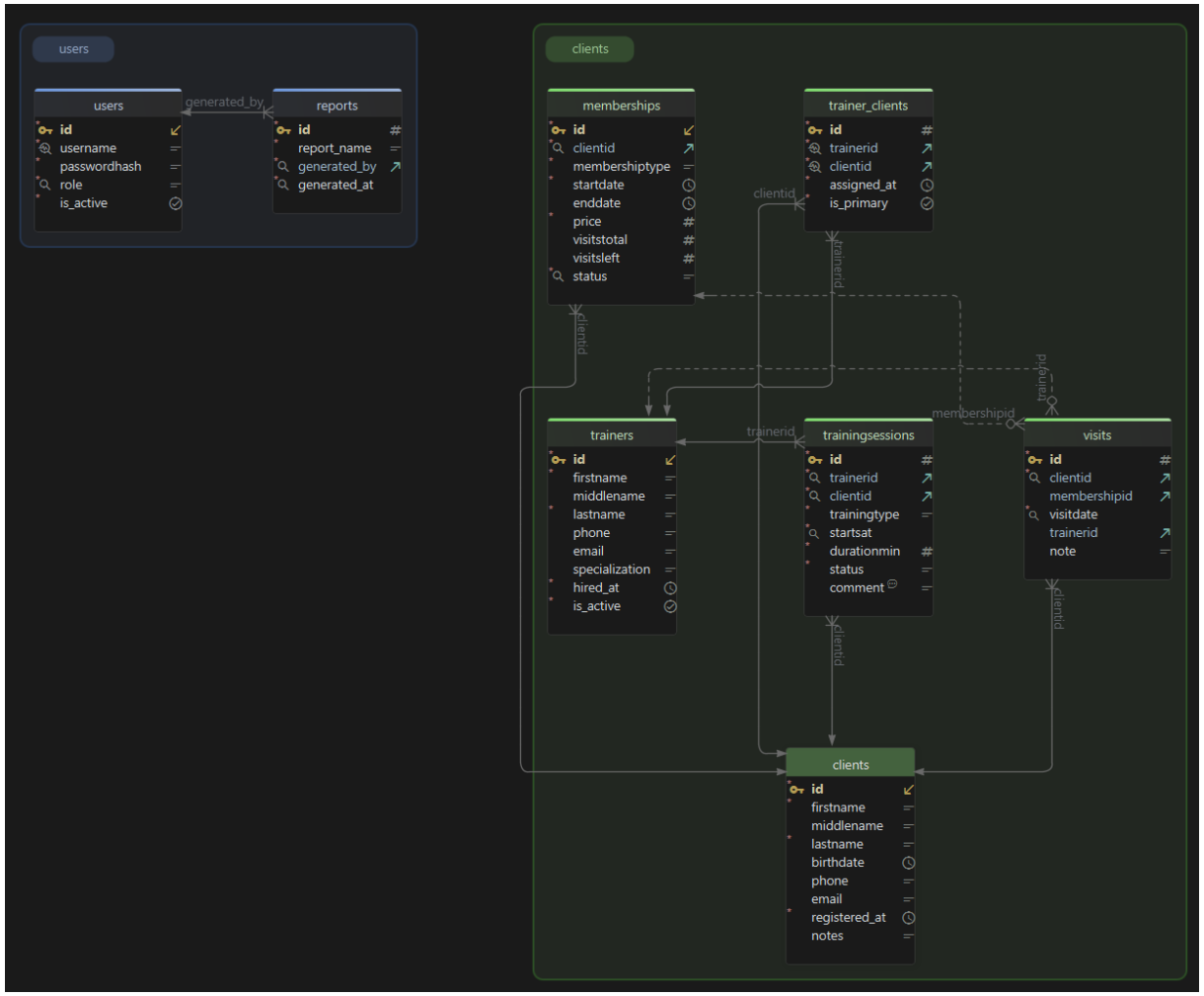

Рисунок 1 - ER-диаграмма база данных

2. Диаграмма использования (Use Case Diagram)

Диаграмма вариантов использования описывает, какие действия могут выполнять пользователи системы Gym System и как они взаимодействуют с функциональностью продукта (рис. 2).

В модели предусмотрены два типа роли:

1) Пользователь (User)

Это обычный сотрудник (например, тренер или сотрудник ресепшена), который имеет доступ только к просмотру информации.

Его возможности ограничены операциями чтения, что исключает риск изменения критически важных данных.

Пользователь может:

- выполнять авторизацию в системе;
- просматривать список клиентов и открывать карточку клиента;
- просматривать список тренеров и их данные;
- просматривать абонементы (действующие/истекшие, параметры абонемента);
- просматривать посещения и сведения о привязке посещения к клиенту/абонементу;
- просматривать расписание тренировок и подробности тренировки;
- просматривать отчёты и аналитику (сводные показатели, таблицы);
- выполнять поиск и фильтрацию записей в таблицах.

Таким образом, пользователь имеет доступ только к информационному уровню системы.

2) Администратор (Admin)

Администратор обладает полным набором прав, включая создание, редактирование и удаление данных. Он имеет доступ ко всем возможностям пользователя, а также дополнительно выполняет операции управления:

- добавление, редактирование и удаление клиентов;
- добавление, редактирование и удаление тренеров;
- добавление, оформление, продление и изменение статуса абонементов;
- назначение тренеров клиентам (управление привязкой тренер–клиент);
- добавление, редактирование и удаление записей о посещениях;
- создание, изменение и отмена тренировок (расписание);
- формирование и выгрузка отчётов.

Таким образом, администратор полностью отвечает за ведение базы данных системы и поддержание её актуальности.

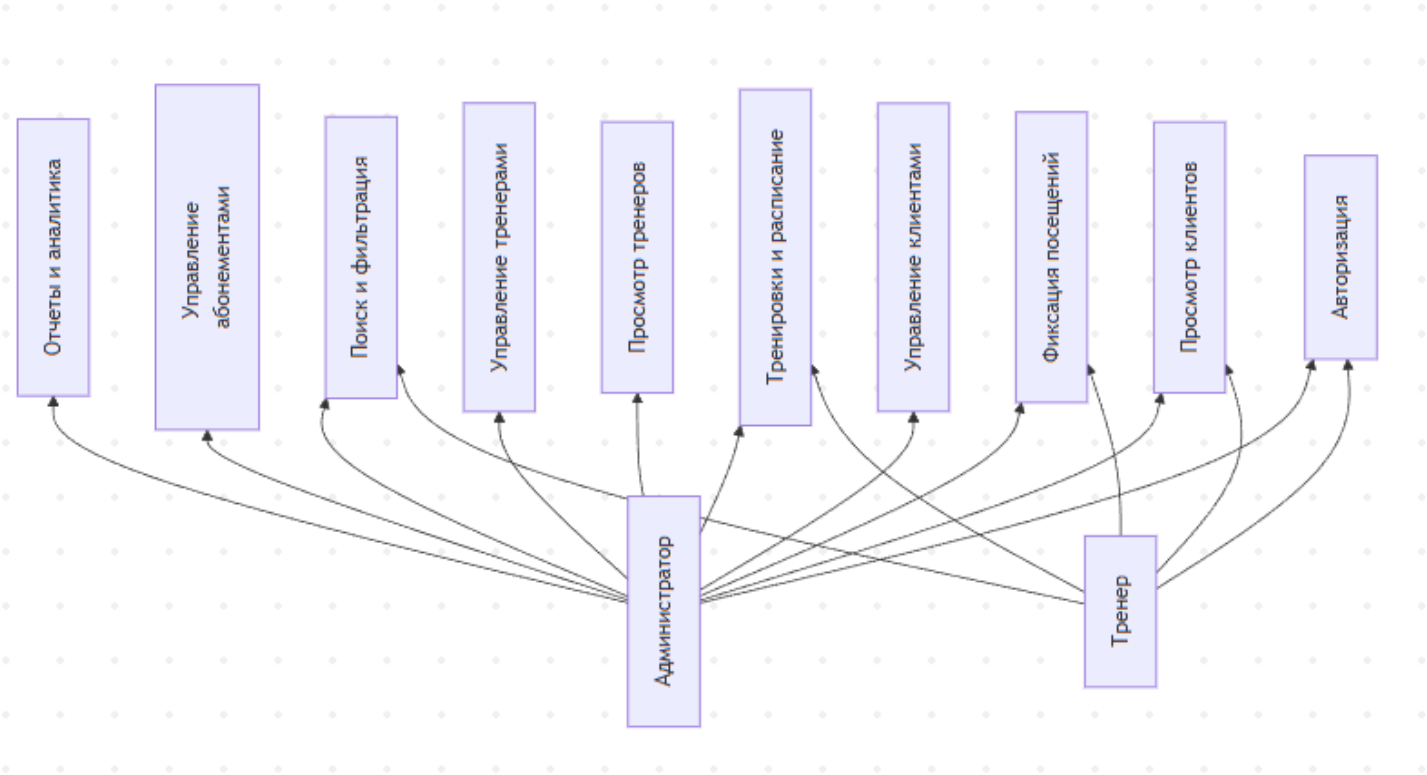

Рисунок 2 - Диаграмма использования

3. Контекстная диаграмма системы (DFD Level 0)

Контекстная диаграмма отображает автоматизированную информационную систему Gym System как единый логический блок и показывает внешние сущности, а также основные потоки данных между ними (рис. 3).

Внешние сущности:

- Пользователь — выполняет запросы на просмотр информации.

- Администратор — выполняет операции управления данными системы.
- База данных PostgreSQL — централизованное хранилище информации.

Основные потоки данных:

1. Пользователь → Система

Запросы на получение данных о клиентах, тренерах, абонементах, посещениях, тренировках и отчетах.

2. Система → Пользователь

Возврат списков записей, карточек объектов и результатов запросов.

3. Администратор → Система

Управляющие действия по созданию, изменению и удалению данных.

4. Система → База данных

Запросы к хранилищу данных.

5. База данных → Система

Возврат данных и подтверждений выполнения операций.

Контекстная диаграмма позволяет определить границы системы, внешних участников и характер взаимодействия между ними, не раскрывая внутреннюю структуру реализации.

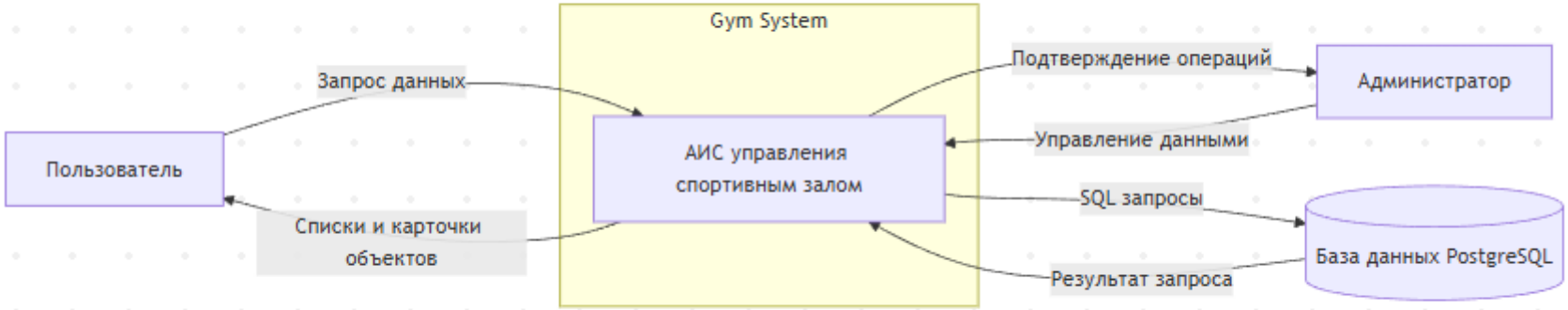

Рисунок 3 - Контекстная диаграмма системы

4. Диаграмма декомпозиции процессов (DFD Level 1)

На уровне декомпозиции автоматизированная информационная система Gym System разбивается на ключевые процессы, каждый из которых отвечает за отдельную часть функциональности системы (рис. 4).

1. Авторизация

Процесс отвечает за проверку учетных данных пользователя.

Обрабатывает ввод логина и пароля, выполняет проверку корректности данных и определяет роль пользователя (Пользователь или Администратор), предоставляя соответствующий уровень доступа к системе.

2. Управление клиентами

Процесс обеспечивает работу с данными клиентов спортивного клуба и включает:

- просмотр списка клиентов;
- получение подробной информации о клиенте;
- добавление, редактирование и удаление клиентов (доступно администратору);
- хранение контактной информации и примечаний.

3. Управление тренерами

Процесс отвечает за учет тренеров и включает:

- просмотр списка тренеров;
- добавление новых тренеров;
- редактирование информации о тренерах;
- удаление или деактивацию тренеров;
- хранение данных о специализации и статусе тренера.

4. Управление абонементами

Процесс предназначен для учета абонементов клиентов и обеспечивает:

- оформление новых абонементов;
- продление и изменение параметров абонемента;
- изменение статуса абонемента (активен, истек и т.д.);
- контроль количества оставшихся посещений.

5. Управление посещениями

Процесс фиксирует факты посещения спортивного зала и включает:

- добавление записей о посещениях;
- привязку посещения к клиенту и абонементу;
- списание посещений с абонемента;
- просмотр истории посещений.

6. Управление тренировками

Процесс отвечает за планирование и учет тренировочных занятий:

- создание тренировок;
- редактирование и отмена тренировок;
- привязка тренера и клиента к тренировке;
- изменение статуса тренировки (запланирована, проведена, отменена).

7. Формирование отчетов

Процесс использует данные системы для формирования отчетности, включая:

- сводные показатели по клиентам и тренерам;
- статистику посещений;
- данные по абонементам;
- аналитические отчеты о работе спортивного клуба.

8. Поиск и фильтрация

Процесс обрабатывает запросы поиска и фильтрации данных:

- по клиентам;
- по тренерам;
- по абонементам;
- по тренировкам и посещениям;
- по статусам и временным периодам.

DFD Level 1 показывает:

- внутреннюю логику системы;
- распределение функциональных обязанностей между процессами;
- движение данных между пользователями, процессами и базой данных.

Диаграмма используется для проектирования архитектуры приложения, API и взаимодействия с базой данных.

Авторизация
Управление клиентами
Управление тренерами
Управление абонементами
Пользователь
Администратор
Управление посещениями
Управление тренировками
Формирование отчетов
Поиск и фильтрация
База данных PostgreSQL

Рисунок 4 - Диаграмма декомпозиции процессов

5. Диаграмма последовательностей

5.1. Диаграмма последовательностей «Просмотр списка клиентов»

Диаграмма отражает пошаговое взаимодействие компонентов системы Gym System при выполнении сценария просмотра списка клиентов (рис. 5).

Участники:

- Frontend (JS + HTML) — клиентская часть (браузер), отображает интерфейс и отправляет запросы.
- API (ASP.NET Core) — серверная часть, принимает запросы и возвращает данные.
- PostgreSQL — база данных, хранит сведения о клиентах.

Сценарий:

1. Пользователь открывает страницу «Клиенты».
2. Frontend инициирует запрос GET /api/clients для загрузки списка клиентов.
3. API получает запрос и формирует SQL-запрос к базе данных.
4. PostgreSQL возвращает результат выборки (список клиентов).
5. API преобразует данные в формат JSON и отправляет ответ на Frontend.
6. Frontend отображает список клиентов в виде таблицы на странице.

Значение диаграммы

Диаграмма последовательности показывает:

- порядок выполнения запросов при отображении данных;
- взаимодействие интерфейса с серверным API;
- обращение API к базе данных и возврат результата;
- потенциальные точки задержки (запрос к БД) и точки ошибок (API/DB).

Такое представление помогает понять работу системы «в динамике» и служит основой для тестирования и оптимизации производительности.

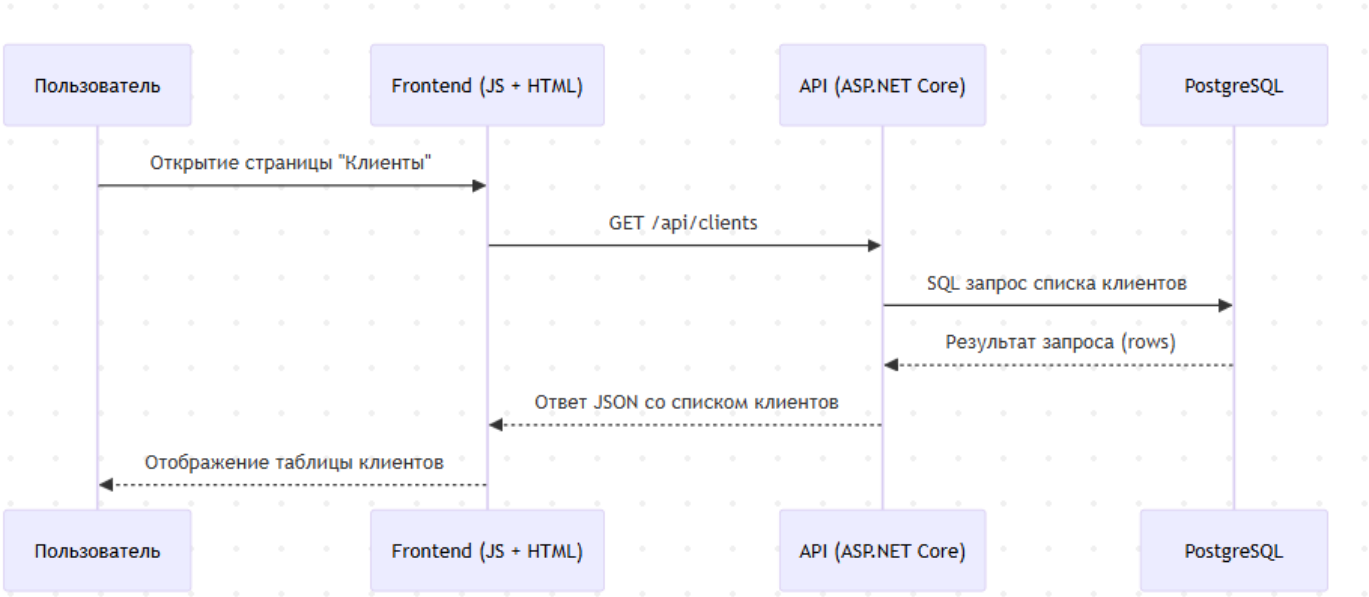

Рисунок 5 - Диаграмма последовательностей «Просмотр списка клиентов»

5.2. Диаграмма последовательности «Авторизация пользователя»

Диаграмма последовательности отражает пошаговое взаимодействие компонентов системы при выполнении сценария авторизации пользователя в Gym System (рис. 6).

Участники:

- Пользователь (или Администратор — процесс одинаковый, различается роль)
- Frontend (JS + HTML)
- API (ASP.NET Core)
- База данных PostgreSQL

Сценарий:

1. Пользователь открывает страницу входа и вводит логин/пароль.
2. Frontend отправляет запрос POST /api/auth/login с учетными данными.
3. API выполняет первичную проверку корректности ввода (валидация).
4. API обращается к базе данных и получает запись пользователя из таблицы users.
5. API сравнивает введённый пароль с passwordhash и проверяет флаг is_active.
6. При успешной проверке система формирует токен/сессию и возвращает ответ 200 OK с ролью пользователя.
7. Frontend сохраняет токен (например, в cookie или localStorage), после чего выполняет переход на главную страницу.
8. При ошибке авторизации API возвращает 401/403, а Frontend отображает пользователю сообщение о причине

отказа.

Значение диаграммы

Диаграмма демонстрирует:

- порядок выполнения запросов при входе;
- проверку учетных данных на стороне API и БД;
- момент назначения роли пользователя и выдачи токена;
- точки, в которых возможны ошибки (неверные данные, заблокированный пользователь).

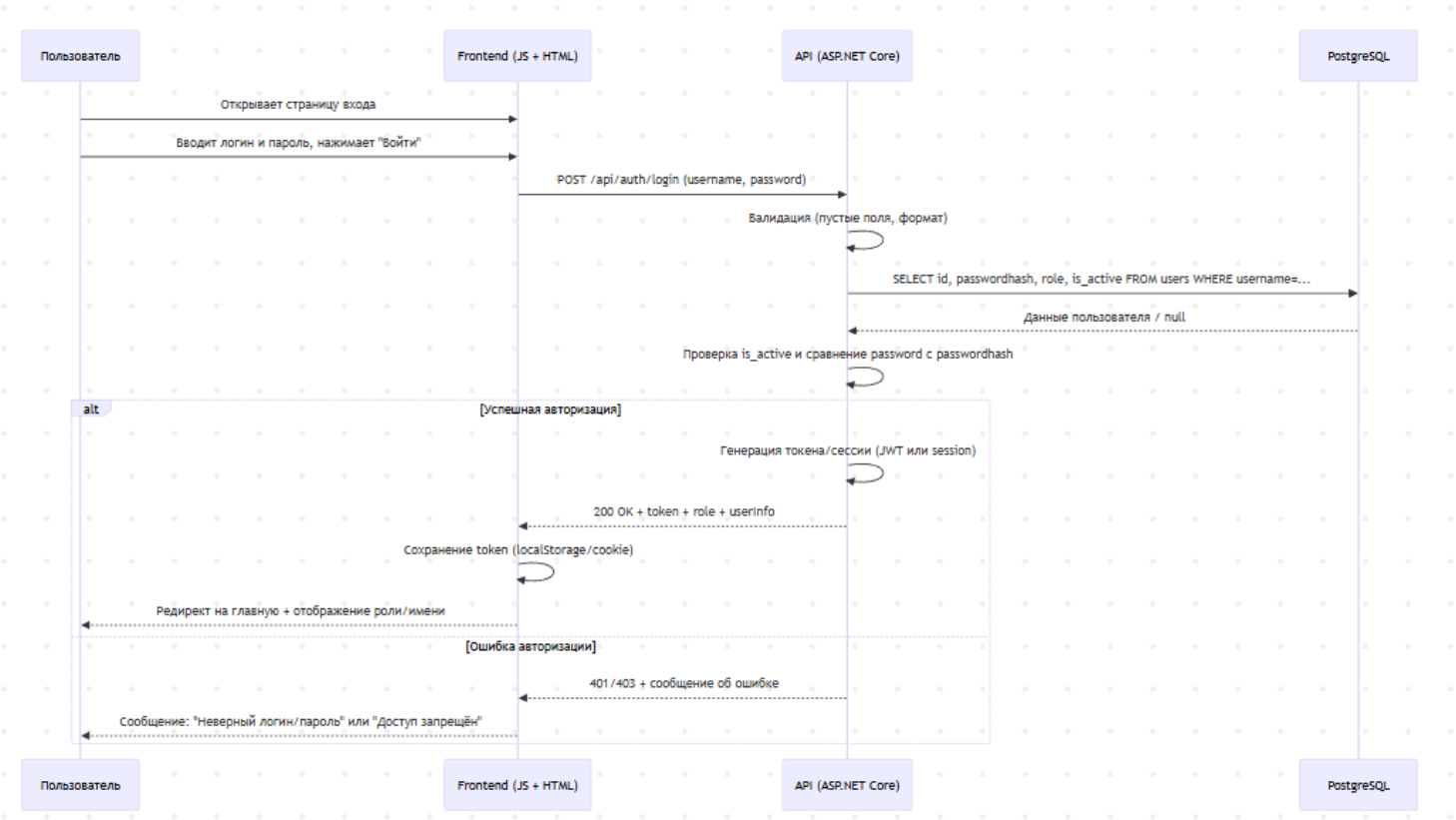

Рисунок 6 - Диаграмма последовательности «Авторизация пользователя»

5.3. Диаграмма последовательности «Добавление клиента»

Диаграмма последовательности показывает, как компоненты системы взаимодействуют при создании нового клиента (рис. 7).

Участники:

- Администратор (или сотрудник с правами редактирования)
- Frontend (JS + HTML)
- API (ASP.NET Core, ClientsController)
- База данных PostgreSQL

Сценарий:

1. Администратор открывает страницу «Клиенты» и нажимает кнопку «Добавить клиента».
2. Frontend отображает форму добавления и принимает введённые данные (ФИО, телефон, email и т.д.).
3. После нажатия «Сохранить» Frontend отправляет запрос POST /api/clients с данными клиента.
4. API выполняет валидацию:
   - обязательные поля заполнены,
   - корректный формат телефона/email,
   - отсутствие дублей (например, по телефону или email — если предусмотрено).
5. API выполняет INSERT в таблицу clients.
6. PostgreSQL возвращает результат операции и id нового клиента.
7. API отправляет ответ 201 Created с созданным объектом (или id).
8. Frontend обновляет таблицу клиентов и отображает нового клиента в списке.

Значение диаграммы

Диаграмма показывает:

- процесс создания сущности Client;
- проверку корректности данных до записи в БД;
- взаимодействие UI → API → DB и возврат результата пользователю.

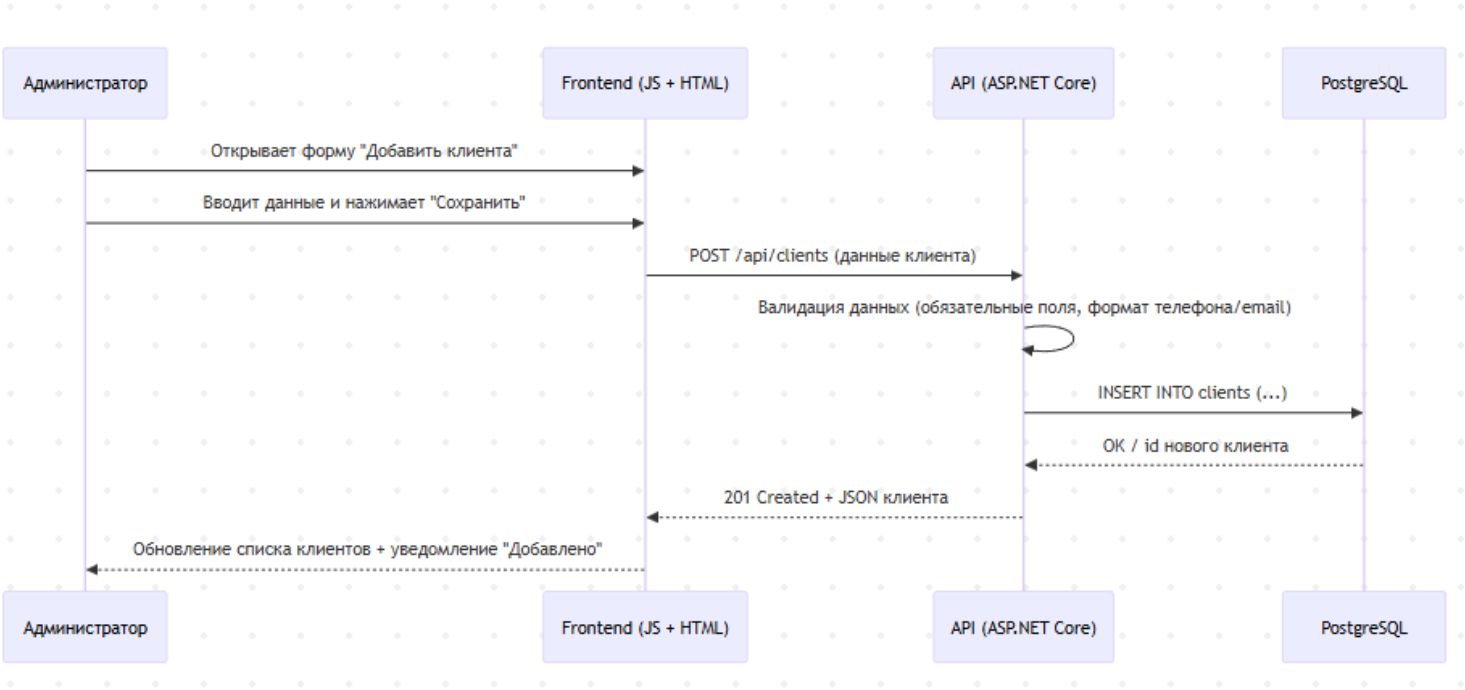

Рисунок 7 - Диаграмма последовательности «Добавление клиента»

5.4. Диаграмма последовательности «Добавление тренера»

Диаграмма отражает сценарий добавления нового тренера в систему (рис. 8).

Участники:

- Администратор
- Frontend (JS + HTML)
- API (ASP.NET Core, TrainersController)
- PostgreSQL

Сценарий:

1. Администратор открывает страницу «Тренеры» и нажимает «Добавить тренера».
2. Frontend показывает форму, в которую вводятся данные (ФИО, телефон, email, специализация, дата найма, активность).
3. Frontend отправляет POST /api/trainers.
4. API валидирует данные (формат контактов, обязательные поля, допустимые значения активности).
5. API выполняет INSERT в таблицу trainers.
6. База возвращает результат и id тренера.
7. API возвращает 201 Created.
8. Frontend обновляет список тренеров и отображает нового тренера в таблице.

Значение диаграммы

Показывает порядок добавления тренера, контроль корректности данных и фиксацию записи в базе.

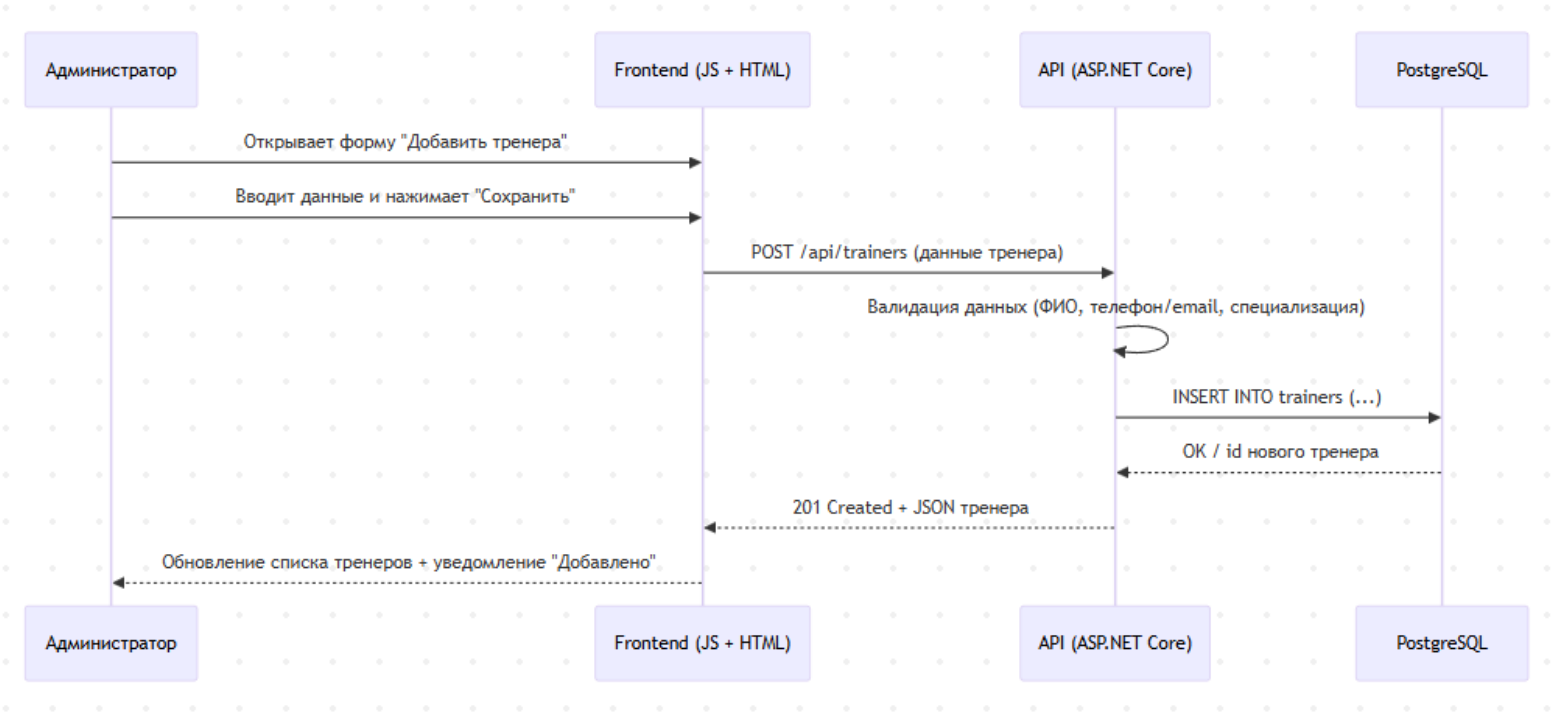

Рисунок 8 - Диаграмма последовательности «Добавление тренера»

5.5. Диаграмма последовательности «Добавление абонемента»

Диаграмма описывает создание абонемента для выбранного клиента (рис. 9).

Участники:

- Администратор / менеджер
- Frontend (JS + HTML)

- API (ASP.NET Core, MembershipsController)
- PostgreSQL

Сценарий:

1. Пользователь с правами управления открывает страницу «Абонементы» и нажимает «Добавить абонемент».
2. Frontend загружает справочные данные (если нужно):
   - список клиентов (GET /api/clients),
   - список типов абонементов (если хранится в справочнике).
3. Пользователь заполняет форму: клиент, тип, дата начала/окончания, цена, лимит занятий (если есть).
4. Frontend отправляет POST /api/memberships.
5. API выполняет проверки:
   - клиент существует,
   - даты корректны (enddate ≥ startdate),
   - цена/кол-во занятий не отрицательные,
   - при необходимости — отсутствие конфликтующего активного абонемента.
6. API выполняет INSERT в таблицу memberships.
7. PostgreSQL возвращает id и подтверждение.
8. API возвращает 201 Created.
9. Frontend обновляет таблицу абонементов и отображает статус (например, «Активен» / «Истёк»).

Значение диаграммы

Фиксирует связь абонемент → клиент, порядок проверок и регистрацию абонемента в БД.

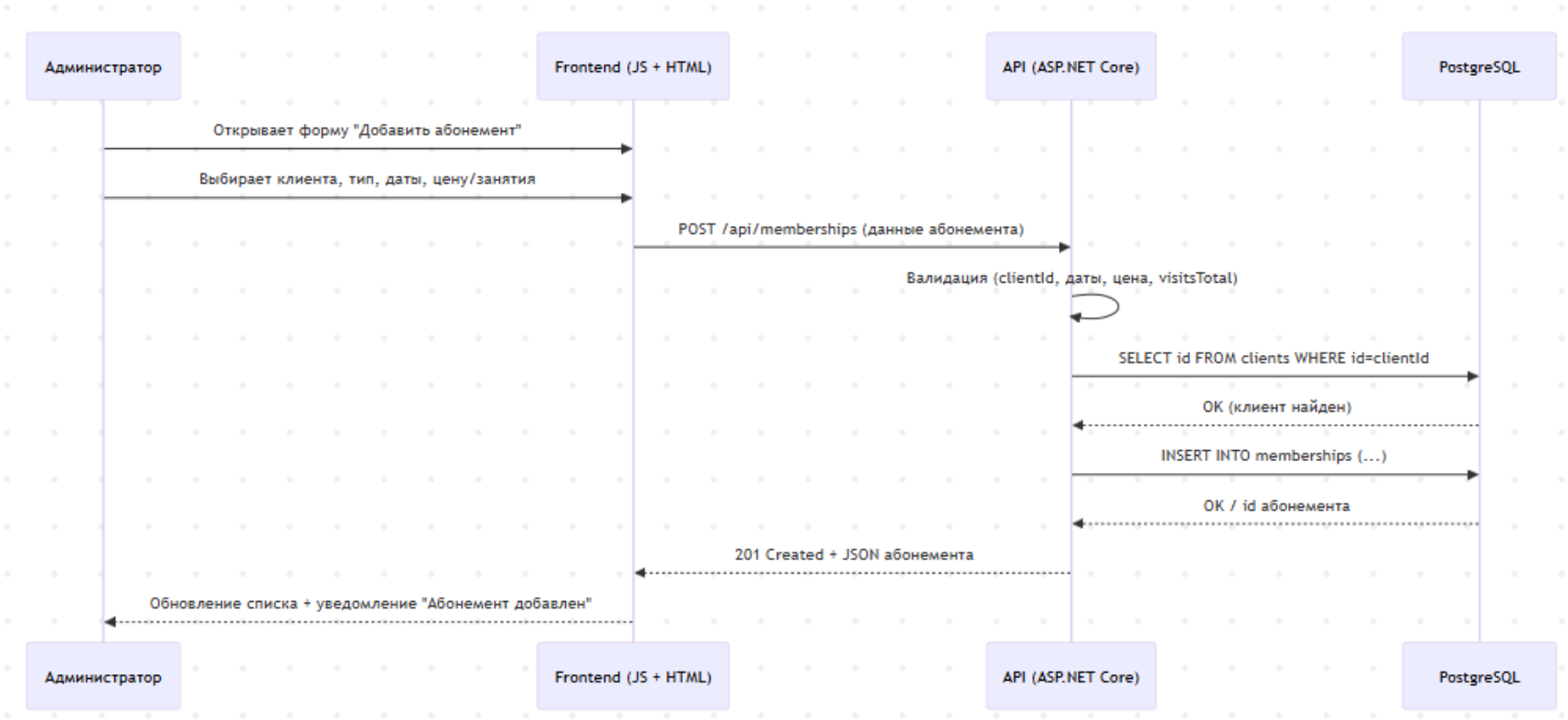

Рисунок 9 - Диаграмма последовательности «Добавление абонемента»

5.6. Диаграмма последовательности «Добавление тренировки»

Диаграмма описывает, как создаётся запись о тренировке (план/сеанс) с привязкой к тренеру и клиенту (рис. 10).

Участники:

- Администратор / тренер (в зависимости от прав)
- Frontend (JS + HTML)
- API (ASP.NET Core, TrainingSessionsController)
- PostgreSQL

Сценарий:

1. Пользователь открывает страницу «Тренировки» и нажимает «Добавить тренировку».
2. Frontend подгружает справочные данные:

- тренеры (GET /api/trainers),
- клиенты (GET /api/clients).

3. Пользователь вводит параметры: тренер, клиент, тип тренировки, дата/время, длительность, комментарий/статус.
4. Frontend отправляет POST /api/trainingsessions.
5. API выполняет проверки:

- тренер и клиент существуют,
- дата/время корректны,
- при необходимости — проверка пересечений по времени (чтобы не было двух тренировок одновременно).

6. API выполняет INSERT в таблицу trainingsessions.

7. База возвращает id.
8. API возвращает 201 Created.
9. Frontend обновляет список тренировок, отображает новую запись.

Значение диаграммы

Показывает, как создаётся сущность TrainingSession, и как контролируется корректность связей «тренер–клиент».

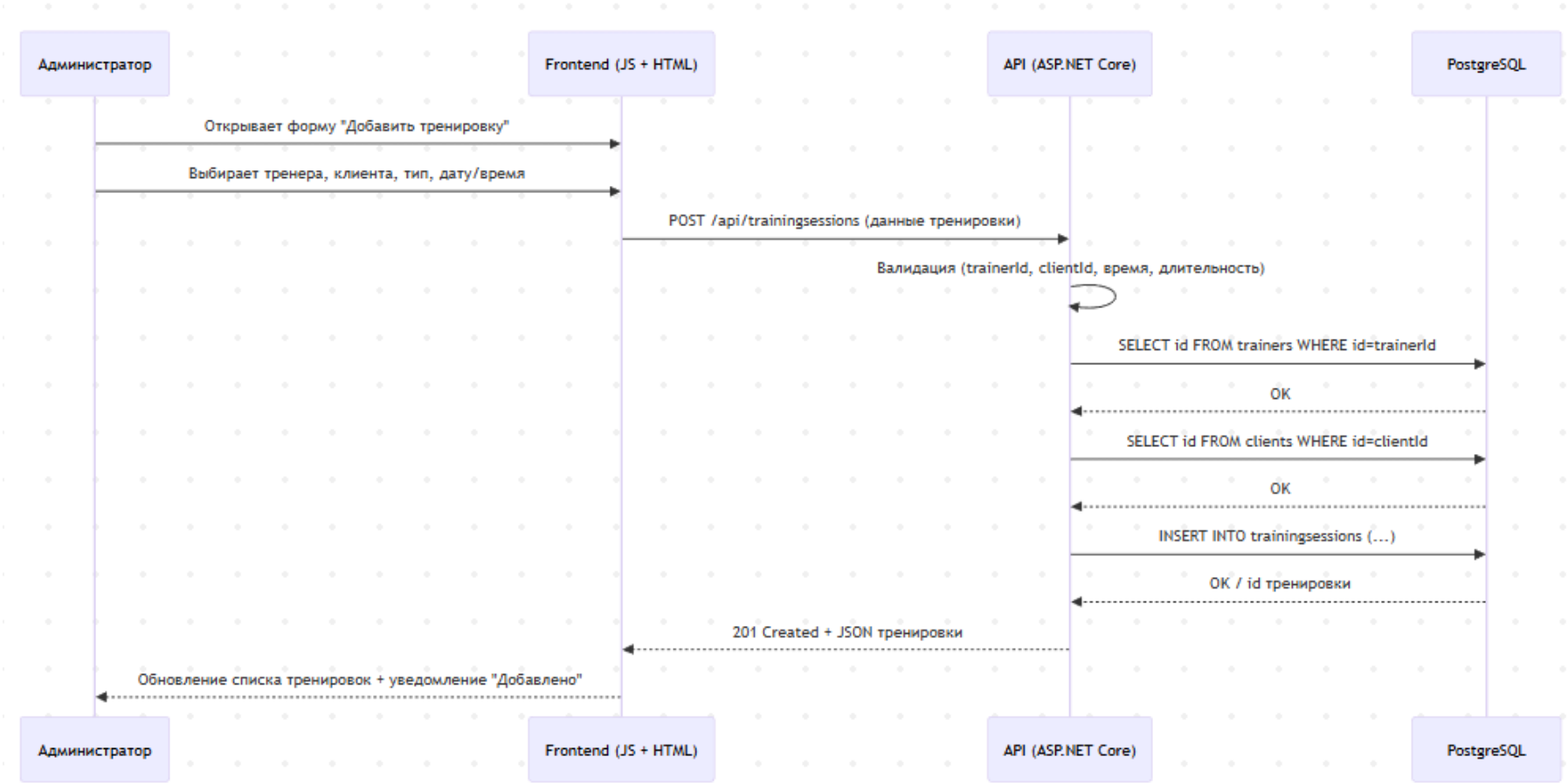

Рисунок 10 - Диаграмма последовательности «Добавление тренировки»

5.7. Диаграмма последовательности «Добавление посещения»

Диаграмма отражает регистрацию факта посещения клиентом (рис. 11).

Участники:

- Администратор / сотрудник
- Frontend (JS + HTML)
- API (ASP.NET Core, VisitsController)
- PostgreSQL

Сценарий:

1. Пользователь открывает страницу «Посещения» и нажимает «Добавить посещение».
2. Frontend загружает данные для формы:
   - список клиентов,
   - список тренеров (если посещение связано с тренером),
   - активные абонементы клиента (если выбирается абонемент).
3. Пользователь выбирает клиента, дату/время, (опционально) тренера, абонемент, примечание.
4. Frontend отправляет POST /api/visits.
5. API выполняет проверки:
   - клиент существует,
   - абонемент (если указан) принадлежит клиенту и активен,
   - если абонемент лимитированный — проверка visitsleft > 0,
   - корректность даты/времени.
6. API выполняет INSERT в таблицу visits.

7. Если посещение привязано к абонементу — API обновляет абонемент:
   - увеличивает visitstotal,
   - уменьшает visitsleft (для лимитированных),
   - при необходимости меняет статус.
8. PostgreSQL подтверждает операции.
9. API возвращает 201 Created (и обновлённые данные абонемента при необходимости).
10. Frontend обновляет список посещений и показатели абонемента.

Значение диаграммы

Диаграмма показывает не только добавление посещения, но и связанную бизнес-логику — корректное списание занятий по абонементу и контроль ограничений.

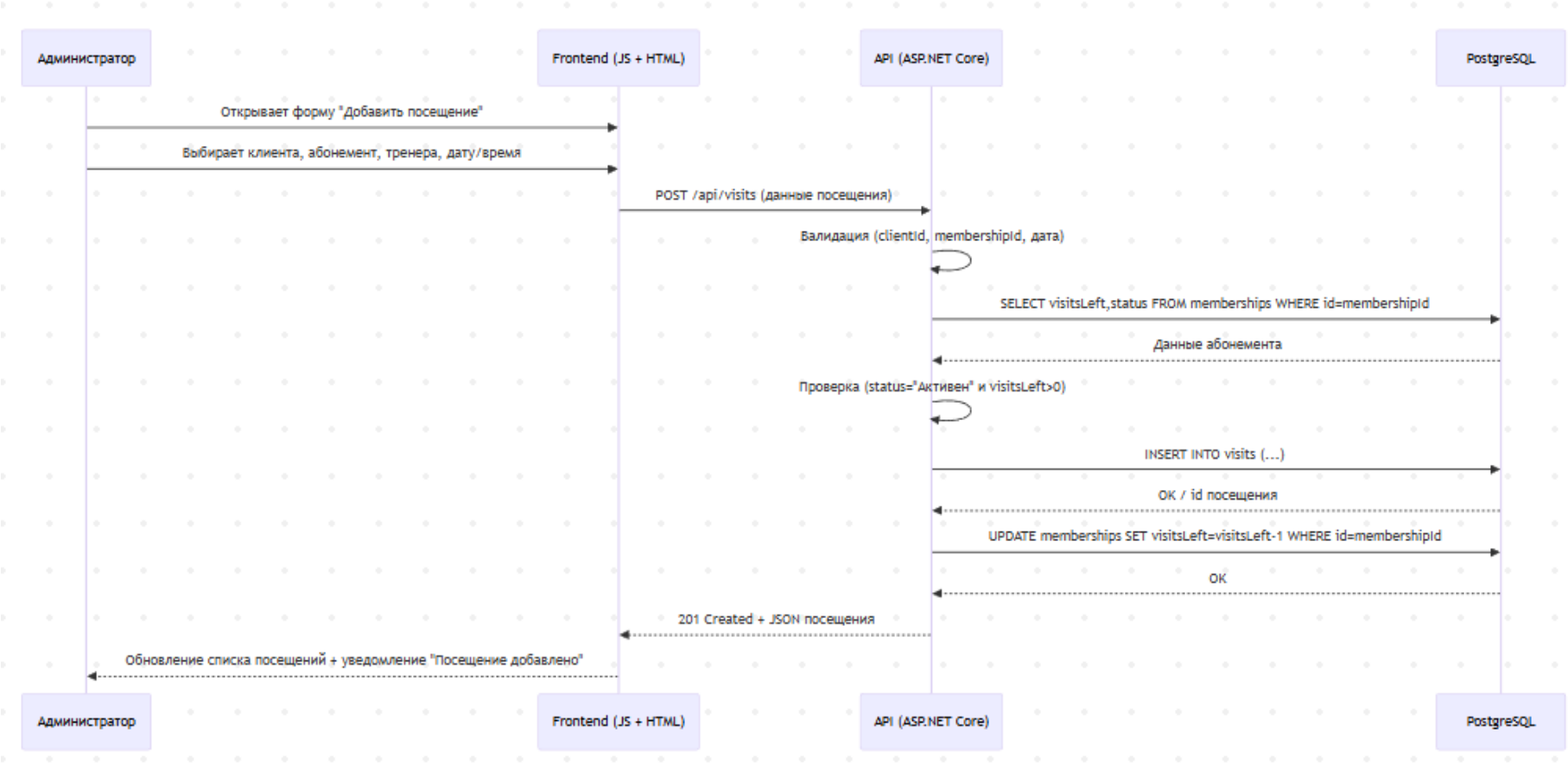

Рисунок 11 - Диаграмма последовательности «Добавление посещения»